ALTER

System Architecture & Security Overview • v1.1.0

Document purpose and scope

Architecture details of Alter.

Alter – Architecture

1. Overview

```
Android APK / Expo Go / Web preview
       │  HTTPS  (EXPO_PUBLIC_BACKEND_URL + /api)
       ▼
FastAPI backend (Python, uvicorn :8001)  ──►  MongoDB (motor, async)
       │
       ├──► Google Gemini API (google-genai)   – chat, capture, voice transcription, summaries
       ├──► Google OAuth / Gmail / Calendar    – user-connected data import
       ├──► Emergent managed Google sign-in   – "Continue with Google"
       └──► Emergent managed email            – feedback + budget alert emails
```

Hosting:

- **Backend:** Emergent deployment, `https://alter-assist.emergent.host` (API only, all routes under `/api`).
- **App:** Android APK built with Emergent Publish; also runs in Expo Go and the web preview.

2. Tech stack

Layer	Technology
Mobile app	Expo SDK 58, React Native 0.88, expo-router (file-based routing), TypeScript
Data and state	@tanstack/react-query, React context (auth), AsyncStorage / SecureStore
Audio	expo-audio (recording; Android records .m4a)
Notifications and alarms	expo-notifications; react-native-notify-kit (Android full-screen intent, SET_ALARM_CLOCK); custom config plugin `plugins/withAlarmActivity.js`
UI	React Native components, react-native-reanimated, Feather icons, theme tokens in `src/theme.ts`
Backend	FastAPI 0.110, uvicorn, Pydantic, motor 3.7 (MongoDB async driver), httpx
Auth	Email + password (bcrypt, JWT bearer tokens) and Emergent-managed Google sign-in
AI	Google Gemini via `google-genai`; models tried in order: 3.5-flash-lite → 3.1-flash-lite-preview → 3.5-flash → 3-flash-preview → 3.8-flash (chat starts with 3.5-flash)
Encryption	cryptography (Fernet) for Google tokens and BYOK keys
Email	Emergent managed email (Resend)

3. Frontend structure (`frontend/`)

```
app/
  _layout.tsx            root stack, providers, AlertsSync
  index.tsx, auth.tsx    entry / sign-in
  (tabs)/home.tsx        one-row capture, day summary, cards
  (tabs)/tasks.tsx       Today/Tomorrow/Later, history, groceries, sync, test alarm
  (tabs)/chats.tsx       conversation list
  (tabs)/memory.tsx      memories by category
  chat/[id].tsx          chat thread (sources, voice)
  profile.tsx, settings.tsx, family.tsx, connect-google.tsx
  admin.tsx, admin-usage.tsx, admin-feedback.tsx   (developer only)
src/
  api.ts                 typed API client + types
  auth.tsx               auth context, Google sign-in
  notifications.ts       schedules alerts/alarms, snooze, test alarm
  alarm.android.ts / alarm.ts   full-screen alarm (Android) / no-op elsewhere
  components/            QuickTask, VoiceSheet, TaskSheet, AlertsSync, FeedbackSheet, AIKeyCard, ui.tsx …
  utils/upload.ts        native-safe file upload
plugins/withAlarmActivity.js   Android manifest changes for lock-screen alarms
app.json                 package com.emergent.alterassist.nez34r, version 1.1.0, versionCode 7, scheme
alter://
```

4. Backend structure (`backend/`)

File	Responsibility
`server.py`	All API routes, models, chat pipeline, capture/voice, Gmail and Calendar sync loop, alarms roll-forward, analytics, seeding
`ai_provider.py`	Gemini routing: user's own key (BYOK) or the app key, model fallback with cooldowns, billing log
`guardrails.py`	Policy prompts, evidence building, grounding and validation, redaction, injection neutralization
`usage.py`	App-wide AI caps, usage counters and events, admin settings, reservations
`budget_alerts.py`	Budget alert and feedback emails
`tests/`	pytest suites and end-to-end scripts

Background jobs (started on app startup):

- the Gmail and Calendar sync loop, every 10 min per connected user;
- a sweep of expired usage reservations.

5. API endpoints (all prefixed `/api`)

Area	Endpoints
Auth	`POST /auth/register`, `POST /auth/login`, `POST /auth/google`, `GET /auth/me`, `DELETE /account`
Profile	`GET/PUT /profile`, `GET /me/usage`
Tasks	`GET/POST /tasks`, `PUT/DELETE /tasks/{id}`, `GET /tasks/history`
Groceries	`GET/POST /groceries`, `PUT/DELETE /groceries/{id}`, `POST /groceries/clear-checked`
Memories	`GET/POST /memories`, `PUT/DELETE /memories/{id}`, `GET /memories/categories`
Chat	`POST /chat` (SSE stream), `GET /conversations`, `GET /conversations/{id}/messages`, `DELETE /conversations/{id}`, `POST

	/messages/{id}/actions/{index}`
Capture	`POST /capture` (typed), `POST /voice` (audio → tasks/groceries), `POST /transcribe` (audio → text)
Home	`GET /dashboard`, `GET /summary/today`
Family	`GET/POST /family`, `POST /family/join`, `POST /family/leave`, `GET /family/board`
Google	`GET /gmail/status`, `POST /gmail/connect`, `GET /gmail/callback`, `POST /gmail/sync`, `POST /gmail/disconnect`
AI key (BYOK)	`GET/PUT/DELETE /ai/key`, `POST /ai/key/test`
Feedback	`POST /feedback`
Admin	`GET /admin/analytics`, `GET /admin/usage`, `PUT /admin/usage/settings`, `GET /admin/billing`, `GET /admin/feedback`, `PATCH /admin/feedback/{id}`, `GET/POST /admin/beta`, `PUT /admin/beta/{email}` (beta is retired)

6. Database (MongoDB collections)

Collection	Purpose / key fields
`users`	email, password_hash, role (admin/user), created_at
`profiles`	user_id, display_name, description, preferences, response_style, timezone, voice_language, family_id
`sessions`	activity sessions for analytics
`tasks`	user_id, title, description, due_date, due_time, priority (urgent/high/medium/low), status (todo/completed), category, recurrence, alarm, source (manual/chat/voice/gmail/calendar), source_text, assigned_to, family_id, confidential
`groceries`	family or user scope, name, quantity, checked, created_at
`memories`	user_id, content, category, sensitive flags, timestamps
`conversations`, `messages`	chat threads and messages (role, content, actions, sources)
`families`	name, join code, members
`gmail_accounts`	user_id, email, encrypted tokens, gmail/calendar flags, last_sync_at, error
`gmail_processed`	(user_id, gmail_id), unique, to avoid re-importing
`oauth_states`	short-lived Google OAuth states (TTL)
`ai_credentials`	user_id, encrypted BYOK key, masked key, status
`ai_key_attempts`	rate limiting of key tests
`ai_billing`	per AI call: user_id, feature, route, model, ok, error, timestamp
`usage_counters`, `usage_reservations`, `usage_events`	app-wide AI caps and usage tracking (no content)
`app_settings`, `config_audit`	admin-editable limits and their change history
`budget_alerts`	one budget email per day (dedupe)
`feedback`	user_id, kind, area, message, screen, platform, app_version, status, emailed
`guardrail_events`	per-reply grounding metrics
`capture_results`	idempotency of typed and voice capture
`daily_summaries`	cached "Your day" summaries
`beta_invites`	legacy (beta retired)

7. Configuration (environment variables, values not included)

- **Backend:**
- `MONGO_URL`, `DB_NAME`, `JWT_SECRET`
- `GEMINI_API_KEY`, `GEMINI_MODEL`, `GEMINI_FALLBACK_MODELS`, `GEMINI_ATTEMPT_TIMEOUT_S`
- `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `BYOK_ENCRYPTION_KEY`, `UNIVERSAL_KEY_ACCESS`
- `EMERGENT_EMAIL_KEY`, `ADMIN_EMAIL`, `FEEDBACK_EMAIL`, `BUDGET_ALERT_PCT`, `CORS_ORIGINS`
- **Frontend:** `EXPO_PUBLIC_BACKEND_URL`, plus Expo packager settings.

8. Key flows

1. **Voice:** expo-audio records m4a → `uploadForm` → `POST /voice` → Gemini transcribes → `VOICE_SYSTEM` extracts actions → `execute_actions` writes tasks and groceries → the app refreshes → `AlertsSync` schedules alarms.
2. **Alarm:** task with time → `syncTaskAlerts` → Android full-screen trigger (SET_ALARM_CLOCK) → ringing screen → Done / Snooze / Stop.
3. **Calendar:** connect (OAuth, redirect `https://<domain>/api/gmail/callback`) → encrypted tokens → sync loop every 10 min → events from today to +30 days become tasks.

Security, privacy and operational controls

Alter – Security & Encryption

Version 1.1.0. This document describes how Alter protects user data, what is encrypted, how access is controlled, and the known gaps. It is based on a code review, not a third-party penetration test.

1. Data Alter stores

Data	Where	Protection
Account email, display name	`users`, `profiles`	Per-user access only
Password	`users.password_hash`	**bcrypt** one-way hash (never stored or logged in plain text)
Tasks, groceries, memories, chats	`tasks`, `groceries`, `memories`, `conversations`, `messages`	Per-user (or explicit family sharing) access; confidential tasks never shared
Google OAuth tokens (Gmail/Calendar)	`gmail_accounts`	**Fernet encrypted** (AES-128-CBC + HMAC-SHA256); never returned by any API
User's own Gemini API key (BYOK)	`ai_credentials`	**Fernet encrypted** with a dedicated key (`BYOK_ENCRYPTION_KEY`); only a masked form (`AIza…1234`) is ever shown
Feedback / bug reports	`feedback`	Visible to the user who sent it and the admin only; emailed to the admin
Usage and billing logs	`usage_events`, `ai_billing`, `guardrail_events`	Contain **no message content** – only feature, model, success/error codes, timestamps
Voice recordings	Not stored	Audio is sent to Gemini for transcription and then discarded; only the transcript is kept

Not stored: card/payment data (Alter has no payments), raw Google passwords, audio files.

2. Encryption

Layer	Method
In transit	HTTPS/TLS for all app ↔ server traffic, and server ↔ Google / Gemini / email provider
Passwords	bcrypt with per-password salt
Login sessions	JWT (HS256) signed with `JWT_SECRET`, 30-day expiry, verified on every request
Google tokens	Fernet, key derived from a server secret
BYOK Gemini keys	Fernet with a separate dedicated key
Database at rest	Handled by the hosting platform's MongoDB storage
Secrets (API keys, client secret, JWT secret)	Server environment variables only; never in the app bundle, code repository or docs

3. Authentication & access control

- **Sign-in options:** email + password, or Emergent-managed "Continue with Google" (Alter never sees the user's Google password).
- **Every request** is authenticated server-side; the user id comes from the verified token, **never from the app**.
- **Every database query** filters by the signed-in user's id (or a family the user explicitly joined). Requests for another user's record return 404. Cross-account isolation is covered by automated tests.
- **Brute-force protection:** sign-in locks for 15 minutes after 5 wrong passwords for an email.
- **Admin role:** only the developer account can open analytics, usage, billing and the feedback inbox; checked on the server for every admin endpoint.
- **Account deletion:** Settings → Delete account permanently removes the user's profile, tasks, groceries, memories, chats, Google connection and saved AI key.

4. Google integration security

- Scopes requested: **`gmail.readonly`** and **`calendar.readonly`** only – Alter can never send email, delete email or change calendar events.
- OAuth uses a random one-time `state` (expires in 10 minutes) to block forged callbacks.
- Tokens are encrypted, never sent to the app, and on **Disconnect** they are deleted and revoked with Google.
- Email and calendar text is treated as **untrusted data**: instruction-like text is stripped before it reaches the AI (prompt-injection protection).
- The Google OAuth app is in **Testing** mode: only accounts listed as test users can connect.

5. AI security

- AI answers **only from the user's own data**; no web search, browsing or tools.
- Secret requests (passwords, OTPs, keys, system prompt) are refused **before** any AI call.
- Secret values are redacted from messages before storage and before reaching the AI; memories that look like secrets are blocked.
- A second AI model fact-checks every chat answer; unsupported claims are removed (fails closed).
- Each user's AI request contains only that user's data.

6. Abuse & cost protection

- App-wide AI caps: 3,000 calls/day and 50,000/month; emergency **AI pause** switch for the admin.
- Email alert to the admin at **90 %** of the daily AI cap.
- Feedback limited to 10 reports per user per day.
- Automatic Gemini model fallback so one exhausted model doesn't take the app down.

7. Logging & privacy

- Server logs record error **types** only (e.g. `TimeoutError`), never message content, tokens or keys.
- Analytics exclude internal/test accounts and contain no message content.
- The AI never receives other users' data.

8. Known gaps & recommended fixes

#	Gap	Risk	Recommended fix
G1	Temporary download links (code/docs zips) are public while they exist	Medium	Delete right after download (done for the code zip)
G2	Google-token encryption key is derived from `JWT_SECRET`; one leaked secret would allow both forged sessions and token decryption	Medium	Use a separate random encryption key (like BYOK) and re-encrypt tokens
G3	Sign-out does not revoke the token server-side; a stolen token works until it expires (30 days)	Medium	Add token revocation (token version per user) and shorter sessions with refresh
G4	Login lockout is kept in server memory: resets on restart, not shared across servers	Low–Medium	Store failed attempts in MongoDB with expiry
G5	CORS allows any website origin (tokens are not cookies, so risk is low today)	Low	Restrict `CORS_ORIGINS` to the app's domains, especially if a web version is published
G6	No per-IP rate limit on registration, voice upload or chat	Low–Medium	Add per-IP limits to stop automated abuse of the AI budget
G7	Developer admin email + bcrypt password hash are set in code	Low	Move to environment variables; use a strong unique password and change it periodically
G8	No automated dependency vulnerability scanning	Low	Run `pip-audit` / `yarn audit` before each release

9. Owner security checklist

- [ ] Enable **two-factor login** on your Emergent, Google Cloud and GitHub accounts.
- [ ] Use a strong, unique admin password.
- [ ] Turn on Google Cloud **billing alerts** if billing is enabled for the Gemini key.
- [ ] Keep the Google OAuth app in **Testing** with a test-user list until you are ready for Google's verification review.
- [ ] Never share `.env` values, test credentials or temporary download links.

- [ ] Rotate `GEMINI_API_KEY`, `GOOGLE_CLIENT_SECRET` and `JWT_SECRET` if you suspect a leak (rotating `JWT_SECRET` signs everyone out).

10. Incident response

4. **Suspected key leak:** rotate the key (Gemini in Google AI Studio, OAuth secret in Google Cloud Console), update it in Emergent's secrets, redeploy.
5. **Suspected account takeover:** reset the user's password; rotate `JWT_SECRET` to sign out all sessions if widespread.
6. **AI cost spike:** switch on **Emergency AI pause** in Admin → AI usage.
7. **Google data concern:** the user can Disconnect Google in Profile (tokens deleted and revoked); the admin can delete the account.
8. Record what happened, when, and what was changed.

11. Reporting a security issue

Email the app owner via the in-app **Send feedback** (choose "Bug" → "Other") or the admin email configured in `ADMIN_EMAIL`. Do not post details publicly.